

Completed-Shuffle Reuse: Implementation and Evidence

Companion to the SPIP discussion draft | Author: Unik Dahal

Purpose: Explain the implemented contracts and tested behavior so Core, SQL, source and shuffle maintainers can assess the proposal. This is supporting design material, not an additional proposal or a stable API specification.

1. Implementation baseline and reading map

All implementation descriptions refer to Spark commit [ab0052497c019d047fee2bea4b2dba1674ae5db4](#) in [the current fork](#). The native validation uses a pinned external source runtime and shuffle-provider fork, with revisions recorded in section 8. Iceberg and Celeborn demonstrate integration; the Spark-side concepts do not depend on their names.

Sections	Review focus
2–3	Source identity, accepted maps and persistent discovery
4–5	Local adoption and actual AQE materialization
6–7	Recovery, trust and deployment lifecycle
8–9	Measurements, test layers and remaining validation
10	Reproduction and code navigation

Three separate facts

Computation equivalence: Spark and the source adapter establish whether the current producer has the same certified semantics, decomposition and output format. **Retained materialization:** the provider exposes native output for accepted attempts. **Permission to read:** the deployment must authorize the current attempt's access. A digest, descriptor or recovery-group string cannot substitute for that permission.

The prototype demonstrates the first two in a trusted test deployment. It has leases and provider incarnation checks, but not a production authenticated retry-lineage service. The proposed use is recovery; the current `recoveryGroup` and caller-supplied generation do not enforce that two submissions are authenticated retries of one logical submission.

Important implementation choices

The path uses an immutable filesystem manifest store, tracker-backed native map availability and ordinary `FetchFailed` dispatch. These are the implementation choices to evaluate for upstream integration.

2. Source certification and computation identity

The internal binding associates certified facts with one actual `BatchScanExec`. The relevant signature is:

```
def bind(
  plan: BatchScanExec,
  protocolId: String,
  protocolVersion: Int,
  certificate: Array[Byte],
  certifiedPartitions: Vector[(InputPartition, Array[Byte])])
  : Either[ShuffleRecoveryMissReason, ShuffleRecoverySourceBinding]
```

Partition objects must match `plan.inputPartitions` exactly and in order. The binding copies descriptor bytes and rejects active runtime filters, grouping, empty/mismatched decomposition and oversized data. Planning exceptions propagate. `isCurrent` rechecks plan/partition identity and filters/grouping.

The native adapter captures the connector's actual planning event. Its cached factory revalidates the exchange after AQE transformations. Both source semantics and physical decomposition must match; equivalent rows with different split planning may miss.

This is not a blanket rejection of every class implementing `SupportsRuntimeV2Filtering`: the code checks active runtime filters and actual bound planning state. Broader post-certification partition changes remain outside the demonstrated flow and require review.

Canonical producer contract

`ShuffleRecoveryCertifiedBatchInputs.build(exchange, binding, providerReadFormatId)` returns either a rejection reason or canonical inputs. It admits the reviewed scan/filter/project grammar and transparent execution wrappers, then checks mapper count against the actual dependency. Unsupported operators/expressions fail closed. It is not a generic use of Catalyst's incidental plan hash.

Identity includes source protocol facts, ordered mapper descriptors, supported expression/type semantics, output partitioning, Spark revision and provider read format. The handoff supplies ANSI mode, session timezone, shuffle compression, codec/block size and I/O encryption settings. Those modeled fields do not imply that every Spark expression, extension or connector version has been audited.

Bound in this source path	Current value
Certified partitions	4,096
Individual split descriptor	64 KiB
Combined certificate/decomposition budget	1 MiB
Framed source token	64 KiB
Complete canonical identity	1 MiB

The Iceberg example uses a 40-byte descriptor per mapper: count, ordinal and ordered decomposition digest. The generic encoding still persists per-split descriptors; this is bounded $O(\text{split count})$ metadata, not a new $O(\text{plan})$ -only identity format. The connector remains trusted for the semantic truth of its certificate.

3. Accepted map publication and manifest discovery

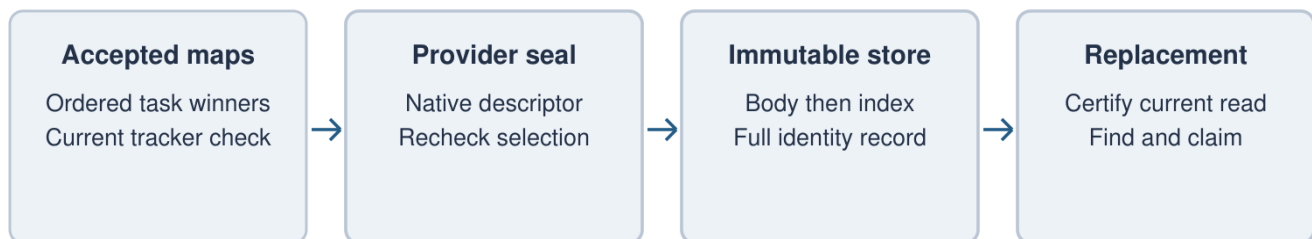
The implemented provider boundary is private and native-format agnostic:

```
trait ShuffleRecoveryNativePublicationProvider {
  def compatibilityId: String
  def seal(
    shuffleId: Int,
    acceptedAttempts: Vector[ShuffleRecoveryMapAttempt]): Vector[Byte]
}
```

The publication context carries recovery group, generation, incarnation, local shuffle ID and canonical identity. Successful task events supply task ID, stage-attempt ID and task-attempt number. A logical map index alone is not the native winner identity.

ShuffleRecoveryNativePublicationBackend requires complete partition-ordered winners matching the certified mapper/reducer counts. It checks tracker agreement before sealing, calls the provider, checks agreement again, then captures map-output estimates under tracker protection. It publishes only a valid bounded descriptor whose format matches the identity.

In the native example, the adapter validates stage/task attempt coordinates before encoding them into the provider's 16-bit attempt fields. This is implemented attempt-coordinate translation, not an unimplemented generic opaque winner-token protocol.



Persistence contract

ShuffleRecoveryManifestStore.publish(manifest) commits an immutable body, then its index reference. Discovery uses findCompatible(recoveryGroup, identity, currentGeneration). Only earlier generations are candidates. It checks group, incarnation, generation, provider compatibility, digest and **full canonical payload equality**. Malformed candidates are skipped individually; overfull discovery namespaces fail closed. Paths and encoded sizes are validated.

The store is an actual filesystem integration point in the PoC. It is separate from native byte storage. Claiming that discovery already belongs to one provider SPI would misdescribe the implementation.

Native manifests carry an opaque provider descriptor and ShuffleRecoveryNativeMapOutput(mapTaskId, reducerBytes). The latter is scheduling metadata, not native byte offsets or authoritative SQL statistics. Native descriptor size is bounded at 5 MiB, total manifest size at 8 MiB, and the dense map/reducer estimate matrix at 131,072 cells.

The pre/post-seal checks do not establish a distributed revocation protocol for an artifact invalidated later. Caller generations are ordering metadata, not a provider-authoritative security fence. Durable treatment of later semantic invalidity remains a review question.

4. Preparation and local scheduler adoption

The configured native adapter discovers a compatible manifest, acquires a provider-backed read binding and offers it against a reservation for the current dependency. Preparation returns a session that the caller retains until completion/cleanup, or a reason to use ordinary execution.

The implemented local installation contract is:

```
trait ShuffleRecoveryNativeInstallation extends AutoCloseable {
  def compatibilityId: String
  def descriptor: Vector[Byte]
  def location: BlockManagerId
  def isCurrent: Boolean
  def install(): Boolean
  def invalidate(): Unit
}
```

Compatibility, liveness, installation and invalidation are local operations. `close()` may perform provider I/O and is deferred by the backend. `offerPrepared` takes ownership even when it rejects an offer. The caller cannot keep using a rejected installation.

Adoption transaction

`ShuffleRecoveryNativeAdoption.beforeFindMissingPartitions` checks the exact dependency object, current reservation, mapper count, existing tracker entry with zero available outputs and current prepared installation. It installs the handle binding, replaces the expected empty `ShuffleStatus`, records adoption and increments the tracker epoch. If adoption does not commit, ordinary execution wins and the prepared binding is invalidated/released.

The restored statuses are `ShuffleRecoveryNativeMapStatus` objects with captured estimates and accepted task IDs. Their synthetic location identifies the binding; native bytes are read through the manager's wrapped handle, not an ordinary block reader pointed at a fabricated executor.

Scope and concurrency limits

The backend associates adoption with the local dependency and binding location/epoch. It does **not** implement exclusive action/query-stage availability outside `MapOutputTracker`. Another consumer of the dependency is therefore a relevant review concern. The native fixture deliberately selects one exchange and does not establish safe automatic reuse across arbitrary shared plans.

Provider preparation finishes before scheduler serialization; remote cleanup is deferred. However, the harness performs discovery and claim work before map-stage submission and may wait for it. There is no demonstrated zero-delay all-miss path. A late or rejected offer cannot overwrite an already running ordinary materialization, but low preparation cost is a performance question still to measure.

The current generic scheduler hooks are `beforeFindMissingPartitions`, `handleFetchFailure`, `consumeWholeStageRetryRequirement` and `isAdopted`. Native managers expose them through `ShuffleRecoverySchedulerBackendProvider`; indexed reference recovery remains available through its resolver. This is an internal PoC integration, not a stable external ABI.

5. AQE lifecycle: implemented and tested

AQE can transform an exchange before materializing its shuffle query stage. Preparing the initial exchange's dependency alone can therefore bind the wrong local object. The prototype now attaches a driver preparation callback as a plan tag and executes it on the actual exchange used for materialization.

```
ShuffleRecoveryExchangePreparation.attach(initialExchange) { actual =>
  // Revalidate certified source and producer for this actual exchange.
  // Attach publication or prepare native adoption before map submission.
}
```

The hook runs from `ShuffleExchangeExec.mapOutputStatisticsFuture` immediately before `sparkContext.submitMapStage(shuffleDependency)` for a nonempty blocking exchange. The source factory retains its original certificate and verifies the final producer; it does not silently certify another scan.

Phase	Actual behavior
Initial planning	Capture the source certificate and attach preparation to the selected exchange.
AQE stage construction	Stage rules can copy the exchange; plan tags carry the preparation callback.
Materialization	Prepare the final dependency, then submit its map stage through the existing path.
Scheduler decision	Install a ready retained binding or submit ordinary maps.
Statistics and reader selection	Existing map-stage completion feeds AQE; AQE selects full/coalesced reads in the tested modes.
Consumption	The wrapped handle routes reads through the native retained reader or the ordinary delegate after invalidation.

Statistics correction

Retained adoption launches no current-driver map tasks, so current shuffle-write metrics remain zero. Treating that as exact zero rows can let AQE infer an empty relation incorrectly.

`ShuffleExchangeExec.runtimeStatistics` now returns `Statistics(conf.defaultSizeInBytes, None)` while adopted. The row count is unknown, not zero.

Captured reducer sizes still feed the existing map-output statistics path as estimates. The PoC does not introduce a separate authoritative-statistics API. Passing coalescing proves this fixture's behavior; it does not certify every statistics-dependent adaptive rewrite.

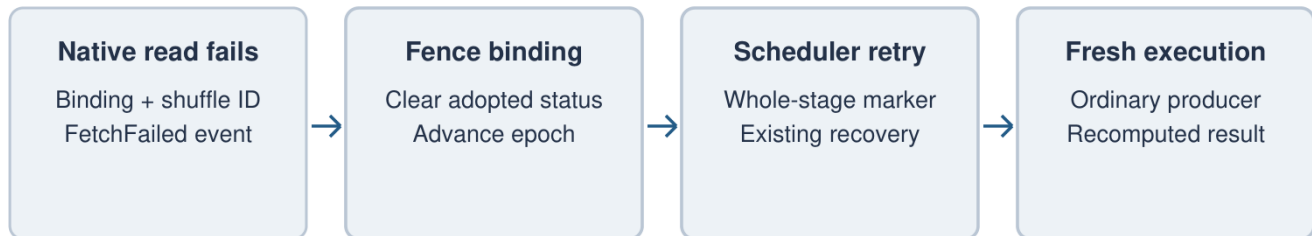
Concrete read coverage

The `full` mode enables AQE with explicit four-way repartitioning and coalescing disabled. The `coalesced` mode uses column-based repartitioning with four configured shuffle partitions and requires an actual range spanning multiple complete reducers. Its final plan contains `AQEShuffleRead coalesced`; merely enabling AQE is insufficient to pass.

The native run also exercises misses, expiry and artifact loss in both adaptive modes. Skew/partial-mapper readers, custom AQE rules, shared stages and arbitrary downstream graphs remain unproven. Materialization continues through the existing query-stage future.

6. Failure dispatch and recomputation

The native reader verifies the current driver binding and holds an independently renewed provider lease. Native failures before or during iterator consumption are translated to `FetchFailedException` with the retained binding location and current shuffle ID. Cancellation preserves ordinary interruption behavior.



Local failure transition

The scheduler routes a native failure to `handleFetchFailure`. For the current dependency and binding, the backend invalidates the handle, removes adopted state, replaces the adopted tracker registration with an empty `shuffleStatus`, clears serialized caches and increments the epoch. Provider release is deferred. A stale location or old event does not invalidate a new binding.

Successful invalidation records a one-shot whole-stage retry requirement. `ShuffleMapStage` consumes and latches it into the existing indeterminate-stage recovery decision. `DAGScheduler` then uses its established stage retry/rollback machinery. The native failure path also avoids treating the synthetic binding address as an ordinary failed host requiring unrelated output cleanup.

Spark's existing recovery rules may retry or abort according to stage state. The proposal must not promise arbitrary action rollback or exactly-once application effects.

What the native fault tests establish

Both artifact loss and lease expiry are injected **after adoption and before the result read**. In every mode, the read observed fetch failure, invalidated adoption, ran one fresh producer map task and returned the baseline 32-row result. These tests validate the native dispatch and recomputation path with AQE full/coalesced readers.

They do not demonstrate a native fault after a user-visible result partition has been accepted, nor revocation during an already-open stream. The surrounding scheduler/reference suites exercise additional boundaries, but those tests are not interchangeable with the exact native scenario. Shared consumers and irreversible actions require their own admission and tests.

Error categories

Incompatible or missing discovery records and rejected preparation become ordinary execution in the harness. Source planning exceptions retain normal query semantics. Native availability errors enter the fetch-failure path. The current native adapter does not expose a separate production corruption/quarantine protocol; do not infer one from local invalidation or lease expiry.

7. Trust, lifecycle and deployment contract

Current operational inputs

The native harness explicitly supplies a manifest root, recovery group, generation, provider format, provider endpoint and lease duration. Producer generation is earlier than replacement generation; publication incarnation distinguishes an artifact. These values support discovery and local lifecycle checks. They are not signed assertions or access credentials.

The source is resolved independently in each driver JVM. Provider-native data and immutable manifests survive driver scratch cleanup. The provider's standalone lifecycle owner outlives the producer. Replacement claims have independent leases; release of one reader must not terminate another. Concurrent replacement JVMs test that property.

Leases and owner restart

The native adapter renews leases outside scheduler execution. A locally expired lease stays fenced even if a delayed renewal later succeeds. The test pauses the lifecycle owner long enough to expire the replacement lease, then resumes the same owner and verifies recomputation. A separate test restarts the owner with a new incarnation and requires rejection of the old candidate.

These tests do not establish replicated lifecycle metadata or transparent provider control-plane failover. Retention policy, storage quotas and cleanup costs remain provider/operator responsibilities. The Spark backend has bounded state and cleanup work; boundedness does not by itself establish production-scale throughput.

Production obligations before enablement

Boundary	Required deployment work
Source access	Preserve normal current source authorization; certify the actual authorized read.
Manifest root	Restrict read/write access and protect canonical source metadata. Filesystem integrity checks are not user authentication.
Retained bytes	Authorize the replacement principal through the provider; a descriptor or matching digest is insufficient.
Retry membership	Decide whether an authorized scope is sufficient or authenticated logical retry lineage is required. The current group string does not enforce lineage.
Encryption	Establish secure cross-attempt key access without copying old application secrets into metadata. The native fixture runs with I/O encryption disabled.
Invalid artifacts	Define provider behavior for known corruption or semantic invalidity, including future discovery/claims. Local invalidation is not durable quarantine.

The mechanism remains generic because Spark validates semantics and scheduling while providers implement their native read/lifetime model. Genericity does not mean every provider satisfies this contract without adaptation. The reference file provider is an in-tree testing implementation; the native adapter demonstrates a different storage path. Public API placement should be decided with existing shuffle-extension maintainers.

8. Native evidence ledger

Spark: ab0052497c019d047fee2bea4b2dba1674ae5db4
Native provider: unikdahal/celeborn, edb413ee3d5e77fbecf43afa7b1a33d6054ab569
Source runtime: apache/iceberg, e76d63584d7f83b102026749e1ae0f91813cb78e

Native run 34881546887 passed all three matrix modes. Artifacts are named native-proof-evidence-off, native-proof-evidence-full and native-proof-evidence-coalesced. Each contains driver properties, initial/final plans, service logs and recorded build information. Dependency artifacts record revisions and checksums.

Driver/control	Count per mode	Target maps	Result
Baseline	1	1	Ordinary execution
Producer; restart producer	2	1 each	Complete native publication
Replacement hit	1	0	Adopted native read
Concurrent replacements A/B	2	0 each	Independent successful claims
Source token; snapshot; producer filter	3	1 each	Incompatible candidate rejected
Missing manifest	1	1	Normal recomputation
Owner restart	1	1	Old candidate rejected
Artifact loss	1	1	Fetch failure then recomputation
Lease expiry	1	1	Fetch failure then recomputation

That is **13 invocations per mode, 39 total**, all returning 32 rows with the same result digest within their mode. Positive native replacements each read 730 remote bytes with zero fetch failures and zero producer maps. Both adaptive modes report a final adaptive plan. Every coalesced-mode invocation reports one merged reducer range.

The common digest is 25a09c0e32dacd10d7ff9c20a605112c38ea0ecaecc0c3bca23f94cce13703b5. Fault cases may have different fetch-failure counts because task timing differs; the asserted outcome is observed failure plus fresh producer execution and exact result equivalence.

Measurement limits

One host, local[2], one source mapper, four reducers and 32 rows. Driver processes are distinct, but executors/services are not distributed across machines. This is functional evidence, not a throughput, cost or latency benchmark. No speedup percentage is inferred from timing fields.

Only this exact native candidate is used for the current claim. Older opportunity studies and historical harness runs are not evidence for the present AQE flow. Subsequent documentation-only commits do not change the tested code, and are not relabeled as the implementation tested by this run.

9. Validation layers and remaining work

Experimental run 34881536279 passed focused Core recovery, focused SQL/AQE and harness coverage, pinned source conformance, and lint/license gates. Its exact-candidate gate passed. These are focused checks, not a claim that every Spark test was run.

Layer	What it contributes
Identity and source binding	Canonical encoding, negative admission, exact planned-read association and connector conformance.
Manifest/publication and Core recovery	Record validation, accepted winner selection, reservations, tracker invalidation and scheduler integration.
SQL exchange preparation	New suite checks preparation of the actual AQE exchange once, final plan identity and real coalescing.
Native process matrix	Concrete source/provider integration, cross-driver map avoidance, actual native reads and selected failure recovery in all three AQE modes.

What still needs proof

- Representative workload value: driver retry frequency, cost-weighted eligible work, retention hit rate, preparation delay and total operational cost.
- Multiple hosts/executors, many mappers/reducers, speculation and stage retries in the complete native integration, not only component fixtures.
- Explicit consumer eligibility and recovery behavior for shared exchanges, downstream stages and partial result acceptance.
- Additional AQE reader shapes and custom rules; authoritative native statistics if broader adaptive decisions require them.
- Authentication, manifest isolation, encryption and secure cross-attempt provider access in a realistic deployment.
- Provider failure during open streams, lifecycle failover and durable handling of known-invalid materializations.
- Generalized connector conformance and API compatibility beyond the demonstration adapter and test fixtures.

Reproducible opportunity study

Start with real retry incidents rather than queries selected to guarantee reuse. Record the original source and AQE settings; report every exclusion and the cost of the eligible producer. Separate changed semantics from changed physical layout. Include all-miss and first-attempt overhead, not only successful replacements.

Neither the current code nor these documents claims that all those gates must be solved before early discussion. They identify the evidence needed to choose a useful upstream scope. The existing implementation is the baseline for that discussion.

10. Reproduction and implementation index

Use the [tested fork tree](#), not an unpinned moving checkout, to reproduce the recorded result. The native workflow builds pinned source/provider dependencies and runs each mode in its own services and storage. The harness README documents required runtime paths and endpoints.

```
# With pinned runtimes and provider services configured:
export SPARK_RECOVERY_AQE_MODE=full
runner=dev/shuffle-recovery/celeborn-native-spike/cold-process.sh
bash "$runner" /tmp/native-full-new-run
```

```
# Use a separate, new directory and isolated services for coalesced/off.
```

The [native workflow](#) is the complete CI recipe. It runs the service supervisor and the `off`, `full`, `coalesced` matrix. Native validation is tag-triggered on the fork; it is not an ordinary upstream Spark test configuration.

Code review entry points

- [Source binding and certified exchange inputs](#).
- [Canonical producer builder](#).
- [Native publication and manifest/store implementation](#).
- [Native adoption transaction and scheduler backend contract](#).
- [AQE preparation hook and exchange/statistics integration](#).
- [DAGScheduler failure handling and ShuffleMapStage retry marker](#).
- [Native source/provider harness and reader and source conformance adapter](#).

For upstream review, compare these integration points with the intended target branch. The fork is a prototype, and a green experiment is not a substitute for normal API, scheduler and release review.